20.10 — Транзакции в ADO.NET

Конспект: свойства ACID

Транзакция объединяет связанные SQL-операции в одно действие. Принцип «всё или ничего» означает: либо подтверждаются все изменения, либо неподтверждённые изменения отменяются.

Atomicity, атомарность: операции транзакции сохраняются вместе или откатываются вместе.

Consistency, согласованность: после завершения операции сохраняются заданные ограничения и правила целостности базы.

Isolation, изоляция: одновременные транзакции взаимодействуют по правилам выбранного уровня изоляции, чтобы избежать недопустимого влияния друг на друга.

Durability, долговечность: успешно подтверждённые изменения сохраняются, в том числе после перезапуска сервера.

connection.BeginTransaction() начинает локальную транзакцию и возвращает SqlTransaction. Commit() подтверждает изменения, Rollback() отменяет неподтверждённые изменения.

Все связанные SqlCommand используют одно соединение и один объект SqlTransaction. В конструктор команды передают соединение и транзакцию либо назначают свойство Transaction. Commit() вызывают только после успешного выполнения всех операций.

try/catch обрабатывает ошибки. В catch выполняют Rollback(); сам откат тоже может вызвать ошибку и требует обработки. Ввод пользователя лучше завершить до начала транзакции, чтобы не держать блокировки во время ожидания.

Почему две независимые INSERT не гарантируют целостность заказа

Пусть первая команда добавляет заказ в orders, а вторая добавляет его позицию в order_items. Без общей транзакции первая команда может успешно сохраниться, а вторая завершиться ошибкой: например, из-за неверного product_id, нарушения ограничения или разрыва соединения. В базе останется заказ без позиции.

Внешний ключ проверяет, что позиция ссылается на существующий заказ, но сам по себе не гарантирует наличие хотя бы одной позиции у каждого заказа. Общая транзакция объединяет обе операции: Commit() выполняется после их успеха, а при ошибке Rollback() отменяет добавление и заказа, и уже добавленных позиций. Проверки количества и другие правила всё равно нужны.

Ответы на вопросы для самопроверки

1. Что такое транзакция?
Группа связанных операций, подтверждаемых или отменяемых как одно целое.

2. Зачем нужен принцип «всё или ничего»?
Чтобы сбой одной операции не оставлял частично сохранённые данные.

3. Что означает ACID?
Atomicity - атомарность; Consistency - согласованность; Isolation - изоляция; Durability - долговечность.

4. Чем Commit() отличается от Rollback()?
Commit() подтверждает изменения транзакции, Rollback() отменяет ещё не подтверждённые изменения.

5. Почему команды должны использовать один SqlTransaction?
Чтобы они входили в одну локальную транзакцию на одном соединении и подтверждались или откатывались вместе.

6. Что случится, если создать заказ без позиций?
Останется неполный заказ, который не отражает оформленную покупку.

7. Для чего нужен try/catch?
Чтобы обработать ошибку и попытаться откатить неподтверждённые изменения.

8. Почему транзакции должны быть короткими?
Долгие транзакции удерживают блокировки и могут мешать другим пользователям.

Самостоятельное задание: заказ из двух товаров

1. До транзакции получить и проверить customer_id, идентификаторы двух товаров и количество каждого. Цена берётся из базы, а не из произвольного ввода пользователя.

2. Открыть SqlConnection и вызвать BeginTransaction().

3. Внутри try создать команду добавления orders, передать customer_id параметром и назначить транзакцию. После INSERT получить id нового заказа через SELECT CAST(SCOPE_IDENTITY() AS int) и ExecuteScalar().

4. В той же транзакции получить цены выбранных товаров из products и проверить, что товары существуют и доступны. Если проверка не пройдена, перейти к обработке ошибки.

5. Добавить первую позицию в order_items, передав order_id, product_id, quantity и unit_price параметрами. Использовать ExecuteNonQuery().

6. Добавить вторую позицию тем же способом. Обе команды должны использовать это же соединение и SqlTransaction.

7. Только после успешного добавления обеих позиций вызвать Commit() и сообщить о создании заказа.

8. Если добавление второй позиции завершилось ошибкой, Commit() не вызывать. В catch выполнить Rollback(): должны быть отменены заказ и первая позиция. Сообщить об ошибке и освободить ресурсы; возможную ошибку Rollback() обработать отдельно.